

# AEGIS.

TRUST MESH CONSOLE

*Bounded. Revocable. Verifiable.*

Complete Demo, Operator & Manual Testing Handbook

RAZORPAY BUILDATHON

*"How to give autonomous agents financial authority  
without giving them unlimited trust."*

v1.0 · Generated from the current working repository and live application

## Table of Contents

### Understand

- 1 Executive Summary — What AEGIS Actually Is
- 2 The Mental Model
- 3 Complete Feature Catalog
- 4 Workspace-by-Workspace Operator Guide

### Go Deep

- 5 Authority — Deep Explanation
- 6 Missions — Deep Explanation
- 7 Transaction Pipeline — Deep Explanation
- 8 ALLOW / DENY / ESCALATE
- 9 AI / Anthropic — Where Intelligence Actually Lives
- 10 Concurrent Budget Race (Hero)
- 11 Revocation (Hero)
- 12 Evidence / Ledger — Deep Explanation
- 13 Tamper Detection (Hero — the Climax)
- 14 Independent Verifier

### Test It

- 15 Complete Manual Testing Plan
- 16 Failure / Troubleshooting Guide
- 17 Pre-Demo Reset Checklist

### Win the Room

- 18 What Features Can Make Us Win
- 19 Questions Judges May Ask
- 20 What NOT to Say During the Demo
- 21 Presentation Style & Opening
- 22 Final 7–8 Minute Master Demo Script
- 23 2-Minute Emergency Version
- 24 30-Second Elevator Version
- 25 Final One-Page Cheat Sheet

PART 1

# 1. Executive Summary — What AEGIS Actually Is

## The problem

Autonomous AI agents are starting to take real-world actions on someone's behalf — buying things, subscribing to services, paying other agents. The moment an agent can initiate a payment, "who is this agent" (authentication) stops being the interesting question. The interesting question is: **what, specifically, is this agent allowed to do right now, within what limit, granted by whom, and can that be taken back instantly if something looks wrong?** An API key answers the first question. It says nothing about the second.

## What AEGIS actually controls

AEGIS sits between an agent's decision to act and the money actually moving. Every agent holds a cryptographic capability token (not a raw credential) with a maximum amount, currency, allowed categories, and allowed payment rails baked into the token itself. That authority can be narrowed further — delegated to a sub-agent with tighter limits (**attenuation**), and bounded again to one specific, budgeted objective (a **mission**). Every transaction is evaluated against all of these boundaries before anything executes, the outcome is one of three verdicts ( `ALLOW` / `DENY` / `ESCALATE` ), and every step — not just the final outcome — is written to a signed, hash-chained ledger that a completely separate tool can verify without trusting AEGIS's own code.

## The three words, precisely

| WORD       | WHAT IT MEANS IN THIS IMPLEMENTATION, SPECIFICALLY                                                                                                                                                                                                                                       |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bounded    | A capability token's caveats (amount, currency, categories, rails, expiry) are enforced by the token's own cryptographic structure (a Biscuit authorizer check), not a database flag an application could quietly ignore. A mission narrows further, on top of the token, never past it. |
| Revocable  | Revoking one agent marks its token identifier revoked; every descendant token's verification checks its full ancestry, so revoking one node fails every token derived from it on its very next use — no separate cascade step to run.                                                    |
| Verifiable | Every ledger entry is Ed25519-signed and hash-chained to the one before it. A standalone tool ( <code>verifier/</code> ) that imports nothing from AEGIS's own code can independently re-derive whether that history is intact, from only an export file and AEGIS's public key.         |

## A. The 20-second explanation

"AEGIS gives an AI agent a cryptographic permission slip instead of a password — one with a spending cap, a category list, and an expiry baked in. That permission can only get narrower as it's handed down to sub-agents, it can be revoked instantly and the revocation actually sticks, and every decision is recorded in a way that can be checked afterward by a tool that doesn't trust AEGIS itself."

## B. The 60-second explanation

"Once an agent can spend money for you, checking a single transaction against a rule isn't enough — you need to know it can still only spend what it was actually delegated, that you can cut off its authority instantly and have that *actually* propagate to everything it delegated further, and that you can prove afterward nothing was tampered with. AEGIS answers all three. Authority is a signed capability token, not a config value — it narrows every time it's delegated and can never widen. A mission layers a bounded objective and its own budget on top of that token. Every transaction runs through a real decision pipeline — mission budget gate, capability check, risk evaluation — and resolves to allow, deny, or escalate, never a silent default. Revoking an agent cuts off every sub-agent beneath it, instantly, with no cleanup step. And every decision — not just successful ones — is written to a hash-chained, signed ledger that a separate, independent tool can verify without trusting a single line of AEGIS's own code. That's the whole system: bounded, revocable, verifiable."

## Why authentication alone is insufficient

Authentication answers "is this really agent X." It says nothing about whether agent X's *this specific request* — \$380, category `flights`, counterparty `acme-airlines`, right now — is something X was actually delegated to do. A stolen or over-trusted API key is authenticated perfectly and can still do anything the backend allows. A capability token collapses "who" and "what, exactly, up to how much" into one cryptographically checkable object, so the answer to "is this allowed" never depends on trusting an application's own bookkeeping.

PART 2

## 2. The Mental Model



| LAYER                                   | WHAT IT ACTUALLY IS IN THIS CODEBASE                                                                                                                                                                                                                              |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Principal                               | Created at sign-in ( <code>POST /principals</code> ); issues an API key stored locally in the browser. Owns every agent it creates.                                                                                                                               |
| Root authority                          | A Biscuit capability token minted at agent creation ( <code>POST /agents</code> ) — caveats: max amount, currency, categories, rails, expiry.                                                                                                                     |
| Delegated / attenuated authority        | A child token minted from a parent's ( <code>POST /agents/:parentId/attenuate</code> ). Structurally cannot be wider than the parent in any caveat — enforced at mint time, not by convention.                                                                    |
| Mission                                 | Bounded metadata layered on top of a token ( <code>POST /missions</code> ) — its own goal, its own cumulative budget, optional narrower category/counterparty allowlists. Grants no authority itself; the token remains the cryptographic boundary underneath it. |
| Mission gate → capability/policy → risk | The actual order a <code>POST /transactions</code> call runs through — see Part 7. A denial or escalation stops exactly there; nothing later ever runs.                                                                                                           |
| Allow / deny / escalate                 | <code>combineRiskSignals()</code> 's composite verdict — see Part 8.                                                                                                                                                                                              |
| Execution                               | Rail-agnostic dispatch to a registered adapter — <code>mock_x402</code> only in demo mode.                                                                                                                                                                        |
| Evidence                                | Every stage above writes its own signed, hash-chained ledger entry — allow, deny, and escalate are recorded exactly as durably as a success.                                                                                                                      |
| Independent verification                | <code>verifier/</code> — a standalone tool, zero imports from AEGIS's own code, given only an export file and AEGIS's public key.                                                                                                                                 |

PART 3

## 3. Complete Feature Catalog

Every meaningful implemented feature, in one scannable reference. Features marked **HERO** get a full deep-dive later in this handbook (Parts 5–14) — this table is the fast-reference version: what it does, why it exists, where it lives in the UI, which module implements it, and the one-line demo move. "Common misunderstanding" and "what not to claim" for each are folded into the relevant deep-dive section rather than repeated here.

| FEATURE                                                              | WHAT / WHY                                                                                                                     | UI                                                         | MODULE                                                                        | DEMO MOVE                                                               |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Principal / session                                                  | The account layer. Sign-in issues an API key stored only in this browser — no server-side session cookie.                      | Sign-in screen                                             | <code>src/api/routes/principals.ts</code>                                     | Create a fresh principal ID to start clean                              |
| Root authority <b>HERO</b>                                           | An agent's own capability token — the base of the trust chain.                                                                 | Authority → New agent                                      | <code>src/capability/keys.ts</code> , <code>authorize.ts</code>               | Create an agent, point at the chain diagram                             |
| Delegation / attenuation <b>HERO</b>                                 | Mint a narrower child token from a parent's — structurally cannot widen.                                                       | Authority → Select → Attenuate                             | <code>src/capability/caveats.ts</code>                                        | Attenuate a sub-agent, show the narrower cap                            |
| Capability constraints — amount, currency, categories, rails, expiry | The exact caveats a token (or attenuation) can express; each is a hard boundary checked by the Biscuit authorizer.             | New agent / Attenuate forms                                | <code>src/capability/types.ts</code> , <code>caveats.ts</code>                | Fill the form fields, name each one aloud                               |
| Revocation <b>HERO</b>                                               | Marks a token's identifier revoked; live-checked on every subsequent verification.                                             | Authority → Revoke; Security → Delegation & revocation     | <code>src/capability/revocation.ts</code> , <code>state/revocations.ts</code> | Run the Security workspace's before/after scenario                      |
| Ancestor revocation                                                  | A descendant's verification walks its full ancestry — revoking a parent fails every child, no per-child cleanup.               | Same as above (multi-generation chain)                     | <code>src/capability/authorize.ts</code>                                      | Attenuate 2 levels deep, revoke the root, show both fail                |
| Mission creation / objective / budget <b>HERO</b>                    | A bounded goal layered on a token — its own cumulative budget, must be $\subseteq$ the token's own bounds.                     | Missions → New mission                                     | <code>src/state/missions.ts</code> , <code>mission/policy.ts</code>           | Create one, point at "budget — the boundary this mission cannot exceed" |
| Mission-agent binding                                                | Every mission is registered against exactly one agent; referential integrity enforced at creation.                             | Missions → Agent dropdown                                  | <code>src/state/missions.ts</code>                                            | Show the agent field is required, not optional                          |
| Mission expiry / cancellation                                        | A mission has its own <code>expiresAt</code> ( $\leq$ the token's); <code>POST /missions/:id/cancel</code> transitions status. | Missions form / Mission Detail                             | <code>src/api/routes/missions.ts</code>                                       | Mention only if asked — not a headline demo beat                        |
| Atomic budget reservation <b>HERO</b>                                | One SQL <code>UPDATE</code> computes <code>budget - reserved - settled ≥ requested</code> and reserves in the same write.      | Missions Detail (live remaining figure); Security (attack) | <code>src/mission/reservation.ts</code>                                       | Concurrent budget race — see Part 10                                    |
| Transaction composer                                                 | Amount, currency, category, rail, counterparty, purpose, optional mission — the request shape submitted for evaluation.        | Transactions form                                          | <code>src/api/routes/transactions.ts</code>                                   | Fill one field at a time, narrate what each maps to                     |
|                                                                      |                                                                                                                                |                                                            |                                                                               |                                                                         |

| FEATURE                                               | WHAT / WHY                                                                                                                          | UI                                                                  | MODULE                                                                                         | DEMO MOVE                                             |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Mission gate validation                               | First deterministic check on a mission-scoped transaction — status, counterparty allowlist, category allowlist, then budget.        | Decision Inspector (pipeline stage 1)                               | <code>src/mission/policy.ts</code> 's <code>checkMissionGate</code>                            | DENY scenario — see Part 7                            |
| Capability / policy validation                        | Verifies the token's full Biscuit chain against the request — amount, currency, category, rail, expiry, live revocation check.      | Decision Inspector (pipeline stage 2)                               | <code>src/capability/authorize.ts</code>                                                       | Revoked-authority DENY scenario                       |
| Risk evaluation — behavioral baseline                 | Two disclosed heuristics: transaction-rate window, amount-deviation vs. this agent's own history.                                   | Decision Inspector "Behavioral:" line                               | <code>src/risk/baseline.ts</code>                                                              | ESCALATE scenario — see Part 7 & 9                    |
| Risk evaluation — intent judge                        | Pluggable interface; demo mode = disclosed deterministic stand-in, production = real Anthropic call.                                | Decision Inspector "Intent:" line                                   | <code>src/risk/types.ts</code> , <code>anthropicJudge.ts</code> , <code>api/demoMode.ts</code> | Read the rationale text aloud — it names itself       |
| ALLOW / DENY / ESCALATE <b>HERO</b>                   | The one composite verdict every transaction resolves to.                                                                            | Decision Inspector                                                  | <code>src/decision/decide.ts</code>                                                            | One of each, back to back — see Part 8                |
| Execution — mock_x402 rail                            | Self-built mock rail, fixed price catalog, its own tiny internal HTTP server (loopback only).                                       | Decision Inspector "Execution" stage                                | <code>src/rails/mockX402/</code>                                                               | Point at the settlement reference in a real ALLOW     |
| Execution — stripe_test rail                          | Real Stripe test-mode adapter; exists in code, never registered while <code>AEGIS_DEMO_MODE=true</code> regardless of key presence. | Transactions → Rail dropdown (present, non-functional in demo mode) | <code>src/rails/stripeTestRail.ts</code>                                                       | Only mention if directly asked — see Part 9           |
| Live decision feed                                    | Overview workspace's recent-activity stream reflecting real API calls as they happen.                                               | Overview → Recent activity                                          | <code>public/app.js</code> , <code>GET /stream</code> (SSE)                                    | Glance at it between acts, don't dwell                |
| Concurrent budget race <b>HERO</b>                    | 20 real concurrent transaction attempts against one bounded mission budget.                                                         | Security → Concurrent budget race                                   | <code>src/mission/reservation.ts</code>                                                        | Launch attack, read the "ZERO OVERSPEND" banner aloud |
| Ledger — sequence, hash chain, signatures <b>HERO</b> | Every entry: monotonic seq, content hash, previous-entry hash, Ed25519 signature.                                                   | Evidence → Live ledger                                              | <code>src/state</code> 's ledger module                                                        | Refresh, read one entry's hash/prev/sig aloud         |
| Integrity verification / tamper detection <b>HERO</b> | Recomputes the chain; a demo-only route corrupts one stored entry directly, bypassing AEGIS's own write path.                       | Evidence → Integrity check                                          | <code>src/state</code> + <code>demo-tamper</code> route                                        | Verify → Tamper → Verify again — see Part 13          |

| FEATURE                          | WHAT / WHY                                                                                                                                                                            | UI                                                                               | MODULE                                               | DEMO MOVE                                                  |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------|
| Independent verifier <b>HERO</b> | A standalone tool, zero imports from AEGIS, re-derives integrity from an export + the public key.                                                                                     | Not in the dashboard — a separate CLI (mention, run in terminal if time permits) | <code>verifier/</code>                               | Show the export + CLI commands — see Part 14               |
| Demo Mode                        | One env var that swaps the risk judge and the rail set behind their existing interfaces — nothing else changes.                                                                       | Every Decision Inspector's rationale text names it                               | <code>src/api/demoMode.ts</code>                     | Read the intent judge's own rationale aloud                |
| Six-workspace UI / environment   | <b>Product UX, not a technical trust property</b> — six navigable workspaces, each with its own background photograph, crossfade, parallax, and verdict-colored environment reaction. | Top nav bar                                                                      | <code>public/index.html</code> , <code>app.js</code> | Let it run in the background — never the thing you explain |
| Reduced motion / accessibility   | Parallax and idle drift are paused under <code>prefers-reduced-motion</code> ; every interactive element is keyboard-reachable.                                                       | Whole app                                                                        | <code>public/app.js</code>                           | Mention only if asked about accessibility                  |

## PART 4

## 4. Workspace-by-Workspace Operator Guide

### 4.1 — Overview

|                              |                                                                                                                                                                                  |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>               | The thesis at a glance, before anything is selected. First screen a judge sees.                                                                                                  |
| <b>What's on screen</b>      | Wordmark + "Bounded. Revocable. Verifiable." · a 5-cell fact grid (Authority / Mission / Budget / Decision / Ledger — each links to its own workspace) · a recent-activity feed. |
| <b>What to click</b>         | Nothing, initially — this is a look-and-say screen. Later, the fact-grid cells are clickable shortcuts back to each workspace.                                                   |
| <b>On success/change</b>     | Every cell updates live as you act elsewhere — Authority shows the selected agent, Budget shows the mission's remaining figure, Decision shows the last verdict color-coded.     |
| <b>Point out</b>             | That the whole product's claim fits in three words, visible before a single click.                                                                                               |
| <b>Don't waste time on</b>   | The recent-activity feed in detail — glance, move on.                                                                                                                            |
| <b>Visually impressive</b>   | The environment photograph, full-bleed, readable typography over it.                                                                                                             |
| <b>Technically important</b> | Nothing computed here — it's a pure read view over the same state every other workspace writes to.                                                                               |
| <b>Likely question</b>       | "Is this just a mockup?" → <b>SAY</b> — <i>Everything you see updates from real API responses — nothing on this screen is hardcoded.</i>                                         |

## 4.2 — Authority

|                              |                                                                                                                                                                                                                       |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>               | Create agents and watch authority narrow as it's delegated.                                                                                                                                                           |
| <b>What's on screen</b>      | Agent tree (left) · New agent / Attenuate form (left, context-switches) · Authority Flow diagram (right) — a live root-to-selected-agent lineage.                                                                     |
| <b>What to click</b>         | <code>Select</code> on an agent card · <code>Attenuate</code> to open the sub-agent form pre-targeted at that parent · <code>Create</code> to submit · <code>Revoke</code> (with a confirm dialog) to cut off a node. |
| <b>On success</b>            | New card appears in the tree, "Delegated (attenuated)" label on children, narrower cap visibly smaller; Authority Flow redraws with the new node.                                                                     |
| <b>On failure</b>            | Inline <code>#agentError</code> text — e.g. a duplicate agent ID, or an attenuation attempt wider than the parent (rejected before any token is minted).                                                              |
| <b>Point out</b>             | The Authority Flow diagram's ring narrowing visually, and that struck-through categories in the child's card are exactly what was removed, not just "fewer."                                                          |
| <b>Don't waste time on</b>   | Filling every field manually on stage — pre-type common values before the demo starts if you can (see Part 25).                                                                                                       |
| <b>Visually impressive</b>   | The narrowing authority-flow visualization.                                                                                                                                                                           |
| <b>Technically important</b> | Attenuation is enforced by <code>src/capability/caveats.ts</code> at mint time — a wider child is structurally impossible, not merely discouraged.                                                                    |
| <b>Likely questions</b>      | "Can a sub-agent get its authority back widened later?" → <b>SAY — No — attenuation is one-directional by construction; there's no operation that widens an existing token.</b>                                       |

## 4.3 — Missions

|                              |                                                                                                                                                                                                                                       |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>               | Bind a bounded, budgeted objective to a selected agent.                                                                                                                                                                               |
| <b>What's on screen</b>      | Mission list (left) · New mission form (left) · Mission Detail (right) — goal, live remaining budget, authority boundary, expiry, transaction history.                                                                                |
| <b>What to click</b>         | Select an agent first (Authority tab) · fill Mission ID / Goal / Budget / Categories / Counterparties / Expiry · <code>Create mission</code> · click a mission card to open its detail.                                               |
| <b>On success</b>            | Mission card appears "ACTIVE"; Mission Detail shows <code>budget - settled = remaining</code> as a large live figure.                                                                                                                 |
| <b>On failure</b>            | <code>#missionError</code> — e.g. a mission category not present in the agent's own token categories (rejected at creation, before the mission ever exists).                                                                          |
| <b>Point out</b>             | "Budget — the boundary this mission cannot exceed" label text, and that Mission Detail explicitly shows the arithmetic ( <code>800.00 USD budget - 0.00 USD settled = 800.00 USD remaining</code> ).                                  |
| <b>Don't waste time on</b>   | Expiry/cancellation mechanics — real, but not a headline beat.                                                                                                                                                                        |
| <b>Visually impressive</b>   | The live remaining-budget figure updating after a transaction.                                                                                                                                                                        |
| <b>Technically important</b> | A mission cannot express anything wider than its agent's own token — checked at creation ( <code>validateMissionAgainstToken</code> ), independent of the runtime gate.                                                               |
| <b>Likely question</b>       | "What if I set a mission budget bigger than the agent's max amount?" → <b>SAY — Allowed on purpose — a mission's budget is cumulative across many transactions, the token's cap is per-transaction. They're different dimensions.</b> |

## 4.4 — Transactions

|                                   |                                                                                                                                                                                                     |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>                    | Submit a real transaction and watch the full pipeline resolve.                                                                                                                                      |
| <b>What's on screen</b>           | Submit form (left) — amount, currency, category, rail, counterparty, purpose, optional mission · Decision Inspector (right) — verdict badge, reason, and every pipeline stage with its own outcome. |
| <b>What to click</b>              | <code>Simulate</code> (dry-run, no execution) or <code>Execute</code> (real evaluation, executes on allow).                                                                                         |
| <b>On success (ALLOW)</b>         | Green badge, every stage lit <code>ALLOW</code> , an execution stage with a real settlement reference, mission budget bar moves, environment green signal flash.                                    |
| <b>On failure (DENY/ESCALATE)</b> | Red or gold badge, the exact stage that stopped it highlighted, nothing after that stage shown as having run.                                                                                       |
| <b>Point out</b>                  | That a DENY's pipeline visibly stops at one specific stage — "a denial stops exactly where it actually stopped," not a generic rejection.                                                           |
| <b>Don't waste time on</b>        | Re-explaining every form field twice — say it once while filling ALLOW, then just narrate values for DENY/ESCALATE.                                                                                 |
| <b>Visually impressive</b>        | The stage-by-stage pipeline trace, verdict colors matching the environment signal flash.                                                                                                            |
| <b>Technically important</b>      | Policy runs before risk, always — a network-dependent AI call never runs on a transaction that was already going to be denied (see Part 7).                                                         |
| <b>Likely question</b>            | "Does Simulate actually call the same pipeline?" → <b>SAY — Yes — <code>/simulate</code> runs the identical decision logic, it just never reaches execution.</b>                                    |



## 4.5 — Security

|                              |                                                                                                                                                                                                                                         |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>               | Prove two properties under real adversarial conditions: budgets survive concurrency, revocation actually cuts off trust.                                                                                                                |
| <b>What's on screen</b>      | Concurrent Budget Race panel (left) · Delegation & Revocation panel (right).                                                                                                                                                            |
| <b>What to click</b>         | Select an agent first · <code>New attack mission (\$2,000)</code> · <code>Launch attack (20 × \$380)</code> · separately, <code>Run scenario</code> for revocation.                                                                     |
| <b>On success</b>            | Attack: a per-attempt grid (✓/X), live counters, a green "ZERO OVERSPEND — BUDGET HELD" banner, server-confirmed spend figure. Revocation: before/after verdicts at large scale, the second one denied with the real revocation reason. |
| <b>On failure</b>            | These scenarios are designed to demonstrate correct denial, not to fail themselves — a genuine failure here (overspend, or a post-revocation ALLOW) would be a real bug, not an expected outcome.                                       |
| <b>Point out</b>             | The exact numbers (attempts / allowed / blocked / spend) are server-confirmed, re-fetched independently — not just counted client-side.                                                                                                 |
| <b>Don't waste time on</b>   | Explaining SQL internals unless a technical judge asks — the banner and the number say enough.                                                                                                                                          |
| <b>Visually impressive</b>   | Twenty real, concurrent network requests resolving live in front of the room.                                                                                                                                                           |
| <b>Technically important</b> | The reservation is one atomic SQL statement — this is what actually prevents the race, not client-side pacing.                                                                                                                          |
| <b>Likely question</b>       | "What if two requests hit the exact same instant?" → <b>SAY</b> — <i>That's exactly what this demonstrates — 20 fire genuinely concurrently, and the database's own atomicity, not request ordering, decides who wins.</i>              |

## 4.6 — Evidence

|                              |                                                                                                                                                                    |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>               | Show the ledger, verify it, then prove tampering is detectable.                                                                                                    |
| <b>What's on screen</b>      | Live ledger (recent entries — seq, kind, hash, prev, sig, timestamp) · Integrity check (big status banner + Verify/Tamper buttons) · Independent verifier pointer. |
| <b>What to click</b>         | <code>Refresh</code> · <code>Verify chain</code> · <code>Tamper latest entry</code> (demo-mode only) · <code>Verify chain</code> again.                            |
| <b>On success</b>            | Green "✓ HASH CHAIN VERIFIED".                                                                                                                                     |
| <b>On tamper</b>             | Red "X INTEGRITY VIOLATION DETECTED", with the exact corrupted entry named underneath, and the header badge switching to "Ledger tampered."                        |
| <b>Point out</b>             | That the tamper button modifies stored data <i>directly</i> , bypassing AEGIS's own write path entirely — this is not "AEGIS pretending to catch its own edit."    |
| <b>Don't waste time on</b>   | Reading multiple ledger entries aloud — one is enough to show the hash/prev/sig shape.                                                                             |
| <b>Visually impressive</b>   | The status flip from green to red on the same button, live.                                                                                                        |
| <b>Technically important</b> | The independent verifier reaches the same conclusion from a completely separate code path — this workspace's own check is not the only witness.                    |
| <b>Likely question</b>       | "Could someone edit the database <i>and</i> the hash to hide it?" → answered fully in Part 13.                                                                     |

PART 5 — THE HEART OF AEGIS

## 5. Authority — Deep Explanation

Every agent's authority is a **Biscuit** capability token — a signed, self-contained credential whose caveats (max amount, currency, allowed categories, allowed rails, expiry) are checked by a cryptographic authorizer, not by an application trusting its own database. **Root authority** is minted at agent creation. **Delegation** ( **attenuate** ) mints a child token from a parent's. **Attenuation** is the mathematical property that makes delegation safe: a child token's caveats can only be equal to or narrower than its parent's, in every dimension, simultaneously.

### What is mathematically guaranteed vs. what is demo configuration

| GUARANTEED BY THE IMPLEMENTATION                                                                                                                                                                                                                                                        | DEMO CONFIGURATION (COULD BE SET DIFFERENTLY)                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A child token can never have a larger <code>maxAmountMinorUnits</code> , an added category not in the parent's list, an added rail not in the parent's list, or a later expiry than its parent. Enforced in <code>src/capability/caveats.ts</code> before a child token is ever minted. | Which specific amounts/categories/rails you type into the demo form (2000 → 800, <code>flights,hotels,software</code> → <code>flights</code> ) — these numbers are illustrative, not fixed by the code. |
| Revoking a token makes every descendant's <i>next</i> verification fail — this is checked live, not by a background cleanup job.                                                                                                                                                        | How many generations deep you demonstrate (one attenuation vs. three).                                                                                                                                  |
| A token's caveats are checked by the Biscuit authorizer against the actual transaction request — a category or rail not in the token's own allowlist is a hard authorization failure.                                                                                                   | Whether you show a capability-layer denial (no mission) or a mission-layer denial (mission gate fires first) — see Part 6 for which wins.                                                               |

### Worked example

Root — agent travel-agent

\$2,000 max · flights, hotels, software · stripe\_test, mock\_x402

↓ attenuate

Delegated / Attenuated — agent flights-only

\$800 max · flights only · (hotels, software struck through)

↓ bind to

Mission — mission-flights

\$800 cumulative budget · category: flights · counterparty: acme-airlines

↓ transaction attempt

Transaction — \$380, flights, acme-airlines → 

ALLOW

Every number above is a real, reproducible value from this handbook's own verification pass (see Part 25) — not a hypothetical.

### Revocation propagation

Revoking `travel-agent` (the root, above) does not touch `flights-only` 's stored token at all — nothing is edited, deleted, or pushed to it. What changes is that `flights-only` 's *next* verification includes checking its ancestor chain, and the root is now marked revoked. That single check is what fails — instantly, for every descendant, with no enumeration of "which sub-agents does this affect."

PART 6

## 6. Missions — Deep Explanation

### Why missions exist if capabilities already exist

A capability token bounds what an agent could ever do, in general, for its entire lifetime — its ceiling. A mission bounds what an agent is doing *right now, for one objective* — a cumulative budget spent across possibly many transactions, plus optional narrower category/counterparty restrictions, with its own expiry. An agent with a \$5,000 standing token might be given five different \$500 missions over time, each one auditable and revocable independently of the others and of the token itself. Without missions, every transaction would only ever be checked against the same broad, unchanging ceiling — there would be no way to express "for this specific purchase run, no more than \$800, only flights, only this vendor."

Which mission limits exist

- **Budget** — a cumulative cap ( `budgetMinorUnits` ), enforced as `spent + reserved + requested ≤ budget` .
- **Currency** — must exactly match the underlying token's currency.
- **Allowed categories** — optional; if set, must be a subset of the token's own categories.
- **Approved counterparties** — optional; missions can restrict counterparties even though the token itself has no counterparty concept at all.
- **Expiry** — must not be later than the token's own expiry.

Missions do not declare a rail restriction — rail allowlisting remains entirely the token's concern ( `src/mission/policy.ts` 's own doc comment is explicit about this).

What if the mission allows something the token doesn't — or vice versa?

**EFFECTIVE PERMISSION = INTERSECTION OF BOUNDARIES**

This is enforced two ways, not one. **At mission-creation time**, `validateMissionAgainstToken()` rejects outright any mission whose currency doesn't match, whose categories aren't a subset of the token's, or whose expiry is later than the token's — so a mission wider than its token cannot even be created. **At transaction time**, both checks still run independently anyway (mission gate, then capability/policy) — so even if a mission's stored constraints were somehow looser than the token (they structurally can't be, but the runtime doesn't assume that), the token's own check still applies on top. There is no code path where a transaction is allowed because the mission alone said yes; both boundaries must agree.

Which one is checked first — and why it's not just "either"

The mission gate ( `checkMissionGate` ) runs *before* `decideTransaction` 's capability/policy check, in the route handler. Order: mission status → counterparty allowlist → category allowlist → budget, then capability/policy, then risk. This is deliberate: the mission gate is the cheapest, purely local check (no cryptographic verification, no network), so a mission-scoped denial short-circuits before the more expensive capability check ever runs.

PART 7

7. Transaction Pipeline — Deep Explanation

| STAGE                         | INPUT                                              | CHECK                                                                                                                           | POSSIBLE FAILURE                                                                |
|-------------------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 1. Mission gate + reservation | The mission (if attached), the candidate amount    | Status active? Counterparty allowed? Category allowed? Budget not exceeded? — one atomic SQL write                              | Any of the four → immediate <code>DENY</code> , nothing reserved                |
| 2. Capability / policy        | The presented Biscuit token, the transaction facts | Full ancestry chain valid, not revoked, every caveat (amount/currency/category/rail/expiry) satisfied by the Biscuit authorizer | Any caveat fails or ancestor revoked → <code>DENY</code> , reservation released |
| 3. Risk — behavioral baseline | This agent's own transaction history               | Rate window (5+ in 60s) and amount-deviation (>3× mean, needs 3+ prior) heuristics                                              | Either flag → forces <code>ESCALATE</code> regardless of intent judge           |
| 4. Risk — intent judge        | Delegated goal, transaction purpose                | Demo: always "consistent." Production: real Anthropic semantic comparison                                                       | "inconsistent"/"ambiguous"/"unavailable" → <code>ESCALATE</code>                |
| 5. Composite verdict          | Both risk signals                                  | <code>combineRiskSignals()</code> — see Part 8's table                                                                          | —                                                                               |
| 6. Execution (allow only)     | The verdict, the rail adapter                      | Rail-specific settlement                                                                                                        | Adapter-level failure — see Scenario F below                                    |
| 7. Ledger write               | Every stage's own outcome                          | Signed, hash-chained append                                                                                                     | Not user-visible as a "failure" — this always happens                           |

**Why the stop point matters:** the Decision Inspector visibly highlights exactly which stage produced the final verdict — nothing rendered after that stage ever ran. A `DENY` at stage 1 never touched the capability layer at all; a `DENY` at stage 2 never ran the (network-dependent) intent judge. This is a real property of the code, not a UI simplification — `decideTransaction` 's own doc comment states the ordering is deliberate and load-bearing precisely so risk can never override a policy denial into an allow.

Six scenarios, exact manual test procedure

**SCENARIO A — VALID TRANSACTION → ALLOW**

**Setup:** agent with token cap ≥ amount, optional mission with budget ≥ amount.

**Input:** Amount 380 · Category `flights` · Rail `mock_x402` · Counterparty `acme-airlines` (must match mission's allowlist if one is attached).

**Expected:** Every pipeline stage green, execution stage shows a real `mockx402_...` settlement reference.

SCENARIO B — OVER MISSION BUDGET → DENY

**Setup:** same mission as above, already has some spend.

**Input:** Amount larger than remaining budget (e.g. 5000 against an \$800 mission).

**Expected:** DENY at the mission-gate stage — reason text: "Transaction would exceed this mission's budget (spent so far: X, requested: Y, budget: Z)". Capability/risk stages never run.

SCENARIO C — WRONG CATEGORY → DENY

**Input, mission attached:** a category not in the mission's `allowedCategories` → mission-gate reason "Category "X" is not in this mission's allowed categories (...)".

**Input, no mission (standing authority only):** a category not in the token's own allowlist → the capability layer's Biscuit authorizer itself rejects it; the surfaced reason is a raw `"Failed check(s): ..."` string derived directly from the failed Biscuit rule, not a hand-written sentence — a real, technically interesting detail: **the token's caveats are enforced by the cryptographic authorizer, not application code translating them into English.**

SCENARIO D — INVALID / REVOKED AUTHORITY → DENY

**Setup:** a valid agent, then revoked (Authority → Revoke, or the Security workspace's scenario).

**Input:** the exact same transaction that succeeded before revocation.

**Expected:** DENY at the capability stage — reason text: "Token (or an ancestor) was revoked at <timestamp>: <reason>".

SCENARIO E — BEHAVIORAL ANOMALY → ESCALATE

**Setup:** a fresh agent with no mission attached (standing authority). Submit **three** transactions at a small, consistent amount (e.g. \$50) to build history.

**Input:** a fourth transaction at more than 3× that mean (e.g. \$600 against a \$50 mean = 12×).

**Expected:** ESCALATE — reason: "Behavioral anomaly detected: Amount (X) is N.Nx this agent's historical average (Y), over the 3x threshold". The Decision Inspector's `Intent`: line still shows `consistent` — it is the **behavioral** layer alone that forced this, independent of the intent judge.

SCENARIO F — EXECUTION / RAIL MISMATCH OR FAILURE

The mock\_x402 rail has a fixed price catalog ( `acme-airlines:flights` = \$380 , `acme-hotels:hotels` = \$210 , `cloudco:software` = \$1500 ). An amount that doesn't match the quoted price for that counterparty/category fails at settlement even after an ALLOW verdict — this is a genuine, reproducible execution-layer failure, not a policy denial. It's a good answer if a judge asks "what happens if the rail itself rejects something policy already approved" — but not a headline demo beat; mention it only if asked.

PART 8

8. ALLOW / DENY / ESCALATE

| OUTCOME               | MEANING                                                                  | TYPICAL TRIGGER                                                                                           | EXECUTES?                               | JUDGE SEES                                                                                                             | HOW TO DEMONSTRATE |
|-----------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------|
| <span>ALLOW</span>    | Every stage passed — mission gate, capability, and risk all clear        | Valid amount/category/counterparty/rail within all boundaries, consistent intent, no behavioral flags     | Yes — dispatched to the rail adapter    | Green badge, all-green pipeline, a real settlement reference                                                           | Scenario A         |
| <span>DENY</span>     | A deterministic boundary was violated — policy said no                   | Over budget, wrong category/counterparty, revoked authority, expired/inactive mission                     | No — stops at the failing stage         | Red badge, the exact failing stage highlighted, everything after it never rendered as having run                       | Scenarios B, C, D  |
| <span>ESCALATE</span> | Policy passed, but risk evaluation flagged something worth a second look | Behavioral anomaly (rate or amount deviation) or an intent judgment of inconsistent/ambiguous/unavailable | No — same as deny, execution never runs | Gold badge, the specific risk reason named, the <code>Intent</code> : and <code>Behavioral</code> : lines both visible | Scenario E         |

ESCALATE, precisely

**Why it exists:** not every risky-looking transaction is actually invalid — an unusually large purchase might be entirely legitimate. Escalate is the verdict for "policy has no objection, but this doesn't look like this agent's normal behavior, so don't execute silently." **How it differs from deny:** a deny means a hard boundary was crossed (budget, category, revoked authority) — deterministic and non-negotiable. An escalate means every deterministic boundary was satisfied, and the risk layer alone raised a flag. **Does it stop execution?** Yes — identically to deny, nothing executes on escalate in the current implementation. **What risk condition triggers it, in the reproducible demo path:** the behavioral-baseline heuristics in `src/risk/baseline.ts` — a rate spike or an amount far outside this agent's own history. **Is Anthropic involved in demo mode?** No — the intent judge in demo mode always returns `"consistent"` ; it is

architecturally impossible for the demo-mode intent judge to be the reason for an escalation. Every reproducible escalation in this environment comes from the deterministic, disclosed behavioral layer, never from AI judgment.

**DO NOT SAY** — Implying that "the AI decided to escalate this" in demo mode — it did not; the demo-mode judge cannot produce that verdict at all.

**DO NOT SAY** — Implying escalation pauses for a human reviewer to click approve/deny somewhere in this build — that workflow is not implemented; escalate currently means "verdict returned, not executed," full stop.

## PART 9

# 9. AI / Anthropic — Where Intelligence Actually Lives

## Demo Mode — what actually happens

`AEGIS_DEMO_MODE=true` swaps the intent judge for `createDemoIntentJudge()` ( `src/api/demoMode.ts` ) — a fixed function that performs no analysis and always returns `verdict: "consistent"` , with a rationale string that says, verbatim, *"This is not a real AI risk evaluation and must never be used in production."* This isn't a UI label bolted on top — it's the literal text returned by the judge function itself, visible in every Decision Inspector.

## Non-demo / intended live mode — where Anthropic is used

Outside demo mode, `createServerIntentJudge()` requires a real `ANTHROPIC_API_KEY` and constructs `AnthropicIntentJudge` ( `src/risk/anthropicJudge.ts` ) — a real model call comparing the agent's delegated goal against the transaction's stated purpose, returning a structured consistent/inconsistent/ambiguous verdict with a rationale. This code path exists and is tested ( `anthropic-judge.live.test.ts` , excluded from the default `npm test` run because it needs a real key) — it has simply not been exercised with a real credential in this environment.

## What the LLM/risk judge evaluates — and what it never touches

**Probabilistic judgment may assist risk/intent evaluation, but deterministic authority and budget enforcement are not left solely to the model.**

The intent judge — real or demo — only ever answers "does this transaction's stated purpose look consistent with the agent's delegated goal." It has no path to widen a capability, approve an over-budget transaction, or override a policy denial. Mission budget math ( `src/mission/reservation.ts` ), capability/caveat verification ( `src/capability` ), and revocation checks ( `src/capability/authorize.ts` ) are pure, deterministic, synchronous code — no model call, no network dependency, no possibility of a hallucinated exception. `decideTransaction()` 's own ordering guarantees this: policy runs first and can only make the outcome stricter for risk to act on, never the reverse.

This is safer specifically because a model's judgment is probabilistic and can be wrong in either direction (too permissive or too strict) — keeping the actual money boundary (amount, category, revocation) in deterministic code means a bad or manipulated model output can, at worst, escalate a transaction for review; it cannot authorize spend outside the token's own bounds.

## Why demonstrate in deterministic mode — said as a deliberate choice, not an apology

- **Reproducibility** — the same input always produces the same verdict, so a rehearsed demo behaves identically every time.
- **No external dependency** — no network call, no API-key provisioning, nothing that can fail for reasons outside AEGIS's own code during a live presentation.
- **No provider latency risk** — a demo running on a conference room's wifi shouldn't be at the mercy of a third-party API's response time.
- **No real money movement** — the mock rail is self-built and loopback-only by construction; nothing here could accidentally charge a real card even if misconfigured.

**SAY** — *We're demonstrating in a mode that removes every external dependency on purpose — so what you're seeing is entirely the software pipeline, not the reliability of a third-party API on the day we happen to be presenting.*

## PART 10 — HERO TECHNICAL SECTION

## 10. Concurrent Budget Race

### The attack, and why naive systems fail it

A spending limit implemented as "read the current spend, check it against the budget, then write the new spend" has a gap between the read and the write. If twenty requests hit that gap at nearly the same instant, every one of them can read the same "budget not yet exceeded" state and all twenty can proceed — a classic check-then-write (TOCTOU) race. A budget boundary that only holds under sequential testing hasn't actually been proven to hold at all; concurrency is precisely the condition most systems never load-test for.

### What AEGIS does instead

**Atomic reservation:** `src/mission/reservation.ts`'s `reserve()` is a single SQL `UPDATE` that computes `budget - reserved - settled ≥ requested` and increments `reserved_minor_units` in the *same* write — there is no separate read step for another request to race against. The database's own transaction isolation, not application-level locking or request ordering, is what decides which requests succeed. **State protected:** the mission's `reserved_minor_units` and `budget_minor_units` columns in SQLite ( `src/state/db.ts` ).

### Why sequential testing would not prove the same property

Firing 20 requests one after another, waiting for each response before sending the next, never exercises the actual race window at all — every request would see a fully up-to-date budget by construction. The Security workspace's attack fires all 20 genuinely concurrently (no waiting between dispatches) specifically so the atomic reservation is under real contention, not simulated contention.

### Verified demo numbers (reproduced in this handbook's own verification pass)

```

20 concurrent attempts × $380 each against a $2,000 mission budget
                        ↓
5 allowed · 15 blocked · $1,900.00 spent · $0.00 overspend

```

The exact split (how many of 20 land as allowed) depends only on the mission budget and the per-attempt amount shown on the Launch-attack button — it is recomputed live by the server every run, not a fixed constant in the UI. The invariant that must always hold, regardless of the split: `allowed × amount ≤ budget`, and `allowed + blocked = attempts`.

### Manual test

1. Authority tab → select an agent with token cap  $\geq$  \$2,000.
2. Security tab → `New attack mission ($2,000)` .
3. Click `Launch attack (20 × $380)` .
4. Confirm: the green "ZERO OVERSPEND — BUDGET HELD" banner appears; `allowed × 380 ≤ 2000` ; `allowed + blocked = 20` ; the "Server-confirmed" spend line matches the displayed total exactly.

### What to point at / what to say

**SAY** — *Limits are easy to enforce sequentially. The interesting question is whether they survive contention. Twenty requests can arrive at once — a budget boundary is only meaningful if it survives that, not just the common case where requests politely take turns.*

Point at the per-attempt ✓/X grid first (it's visually immediate), then the four stat numbers, then the green banner's exact spend figure last — that ordering builds from "something happened" to "here's the proof it was exactly correct."

**Why a technical judge should care:** this is a real, load-tested concurrency guarantee backed by an adversarial test in the suite

( `src/mission/__tests__/mission-reservation.test.ts` ) that fires genuinely concurrent calls and asserts on final server state — not a claim resting on the demo alone.

## PART 11 — HERO TECHNICAL SECTION

## 11. Revocation

```

ROOT
↓ attenuate
DELEGATED
↓ attenuate
ATTENUATED
↓ bind
MISSION

```

Revoke the **root**. Every node beneath it — delegated, attenuated, and any mission built on top — loses authority on its very next verification, without any of those descendant tokens being individually touched, edited, or enumerated.

## Manual test

1. Demonstrate a valid transaction succeeding ( `ALLOW` ) under the current authority.
2. Revoke the ancestor (Authority tab → `Revoke` , confirm the dialog — or the Security workspace's scripted scenario).
3. Attempt the exact same transaction again.
4. Confirm `DENY` , reason: `"Token (or an ancestor) was revoked at <timestamp>: <reason>"` .
5. Explain why: the capability layer's verification walks the full ancestry chain on every check — it isn't caching "still valid" from the last time it checked.

## What to say

**SAY** — *Trust is not a permanent credential. It's a live authority relationship — checked fresh on every single transaction, not granted once and assumed to still hold.*

**SAY** — *The transaction didn't change. The authority did.*

**Edge case, disclosed honestly (only if asked):** a transaction already past the capability check and mid-execution at a rail when a revocation lands is not guaranteed-cancellable in the current implementation — named as production hardening work in `docs/SYSTEM_ARCHITECTURE.md` §6, not claimed as solved.

PART 12

## 12. Evidence / Ledger — Deep Explanation

### What gets recorded

| FIELD                     | WHAT IT IS                                                                                                                                                                                                                                                                                |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>seq</code>          | Monotonic sequence number (SQLite <code>AUTOINCREMENT</code> ) — the entry's position in the chain.                                                                                                                                                                                       |
| <code>kind</code>         | What happened — <code>agent_registered</code> , <code>mission_policy_verdict</code> , <code>policy_verdict</code> , <code>risk_verdict</code> , <code>decision</code> , <code>execution_result</code> , <code>revocation</code> , and others — every pipeline stage writes its own entry. |
| <code>data_json</code>    | The full payload for that event — not just a pass/fail flag.                                                                                                                                                                                                                              |
| <code>created_at</code>   | Timestamp.                                                                                                                                                                                                                                                                                |
| <code>prev_hash</code>    | The content hash of the entry immediately before this one — this is the "chain" in hash chain.                                                                                                                                                                                            |
| <code>content_hash</code> | A hash of this entry's own content.                                                                                                                                                                                                                                                       |
| <code>signature</code>    | Ed25519 signature over the entry, produced by the server's ledger signing key.                                                                                                                                                                                                            |

### How verification works

`verifyChain()` walks every entry in sequence and confirms three things simultaneously: (1) each entry's `content_hash` actually matches its stored content — catches direct data tampering; (2) each entry's `prev_hash` matches the actual previous entry's `content_hash` — catches a broken or reordered link; (3) each entry's `signature` verifies against the ledger's known public key — catches a forged entry signed with a different key. All three are independently tested ( `src/state/__tests__` 's "hash-chained ledger — tamper detection" suite covers each category separately).

### Why normal application logging is not equivalent

A normal log is a record the application chooses to write, in a form the same application (or anyone with database access) can also silently edit afterward with no trace. A hash-chained, signed ledger makes any retroactive edit — however small — break a verifiable mathematical relationship, not just "look suspicious." The difference isn't that the ledger is harder to edit at the storage layer (it is, in fact, the exact same SQLite file, editable the same way) — it's that editing it without also correctly recomputing every subsequent hash and re-signing with a key you don't have is what becomes detectable.

**Honest framing, precisely:** this is not "append-only, tamper-proof infrastructure" — the underlying storage can be modified directly (that's exactly what the tamper demo does). What's actually true is narrower and still strong: **modification becomes detectable**, both by AEGIS's own check and by a completely independent tool.

PART 13 — HERO DEMO SECTION, THE CLIMAX

## 13. Tamper Detection

### Exact flow

1. Evidence tab → `Refresh`, then `Verify chain`.
2. Confirm the green "**✓ HASH CHAIN VERIFIED**" status.
3. Click `Tamper latest entry` — a demo-mode-only route that corrupts one stored entry directly in the SQLite file, bypassing every part of AEGIS's own write path (no ledger append, no signature, no application code involved in the corruption itself).
4. Click `Verify chain` again.
5. Confirm the red "**✗ INTEGRITY VIOLATION DETECTED**" status, with the exact corrupted entry named (e.g. `"Entry #81 (decision) was altered directly in storage... content hash and signature were computed over the ORIGINAL content and were never recomputed."`).
6. Explain which relationship failed: the entry's stored `content_hash` no longer matches its (now-altered) content — a content-hash mismatch, the simplest and first of the three checks `verifyChain()` performs.

This tool exists specifically to demonstrate detection — it is a scope-limited, demo-mode-only route ( `src/api/__tests__/demo-tamper.test.ts` covers it directly), not a general-purpose data-editing feature.

### What to say — the climax line

**SAY** — *A normal audit log asks you to trust the system that wrote it. AEGIS lets us check whether that history still agrees with itself — and when it doesn't, it says so, by name, immediately.*

Deliver this line right as the red status appears — it's the strongest single moment in the demo because the room watches the exact same button flip from green to red on real, unstaged data.

PART 14

## 14. Independent Verifier

### Why the dashboard shouldn't verify itself

If AEGIS's own server were the only thing that ever confirmed the ledger's integrity, a compromise of that server's own verification code could hide a compromise of the ledger it's checking — the fox checking its own henhouse. `verifier/` is a genuinely separate tool: it imports **nothing** from `src/`, and depends on nothing running at all — it operates entirely offline against a static export file.

### What's independently verified

- The full hash chain's integrity (re-derived from scratch, not trusting AEGIS's own `verifyChain()` result).
- Every signature, against AEGIS's *public* key only.
- Whether any mission's settled spend, reconstructed independently from the exported ledger entries, ever exceeded that mission's own budget.

### Inputs it needs

An exported ledger file ( `verifier/dist/export/exportLedger.js <db path> <publicKeyHex> <output>` ) and AEGIS's **public** key, printed by the server's own startup log. Nothing else — no database access, no running AEGIS process, no trust in AEGIS's own reporting.

### What success/failure means

Exit code `0` : every entry's hash, chain link, and signature check out, and no mission's reconstructed spend exceeds its budget. Exit code `1` : at least one of those failed — the tool names which. Both are real, observable in a terminal, not narrated claims.

### How to run it / demonstrate it

```
npm run build:verifier
node verifier/dist/export/exportLedger.js ./aegis.db <publicKeyHex> ledger-export.json
node verifier/dist/cli.js ledger-export.json
```

Run this in a terminal *after* the tamper demo, against a fresh export taken post-tamper, if time permits — showing exit code `1` from a tool that has never touched AEGIS's own code is the strongest possible closer for a technical judge, but it is optional and time-boxed; the in-dashboard tamper demo alone already lands the point.

### Positioning

This is the trust property that makes the whole system's "verifiable" claim credible rather than self-reported: AEGIS isn't just checking its own homework — a second, independently-authored program reaches the same conclusion.



## PART 15

## 15. Complete Manual Testing Plan

Every test below was exercised against the live application during this handbook's own verification pass (Part 25). "Reset" means: create a fresh principal (sign-in screen) — the fastest way to a clean slate, since every principal owns an isolated set of agents/missions.

### 15.1 — Authority test cases

| ID | TEST                                   | STEPS & INPUT                                                                                                               | EXPECTED RESULT                                                                           | VALIDATES                                        |
|----|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------|
| A1 | Root creation                          | Authority → fill Agent ID, goal, max amount 2000, categories <code>flights, hotels, software</code> → Create                | New card in tree, "Agent authority" role                                                  | Root token minting                               |
| A2 | Delegation / attenuation               | Select the root → Attenuate → narrower fields (800, <code>flights</code> ) → Create                                         | Child card, "Delegated (attenuated)" role, nested under parent                            | Attenuation mint path                            |
| A3 | Narrower max amount enforced           | Attenuate with max amount <i>larger</i> than the parent's (e.g. parent 800 → child 5000)                                    | <code>#agentError</code> : attenuation rejected before a token is minted                  | <code>caveats.ts</code> 's amount check          |
| A4 | Narrower categories enforced           | Attenuate with a category not in the parent's list (e.g. parent <code>flights</code> → child <code>flights, crypto</code> ) | Rejected — added category not allowed                                                     | <code>caveats.ts</code> 's category subset check |
| A5 | Rail restriction enforced              | Attenuate with a rail unchecked in the parent's set, then checked in the child's                                            | Rejected — added rail not allowed                                                         | <code>caveats.ts</code> 's rail subset check     |
| A6 | Child cannot exceed parent (composite) | Any single dimension wider than the parent                                                                                  | Rejected — attenuation is all-or-nothing narrower                                         | Structural narrowing guarantee                   |
| A7 | Revoked authority fails                | Revoke an agent → immediately retry a transaction that worked before                                                        | <code>DENY</code> — "Token (or an ancestor) was revoked..."                               | Live revocation check                            |
| A8 | Ancestor-revoked child fails           | Two-generation chain, revoke the root, attempt a transaction on the grandchild                                              | <code>DENY</code> , same reason, even though the grandchild's own token was never touched | Ancestry-walk verification                       |

### 15.2 — Mission test cases

| ID | TEST                                             | STEPS & INPUT                                                                            | EXPECTED RESULT                                                                                              | VALIDATES                                      |
|----|--------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| M1 | Valid mission                                    | Missions → fill ID, agent, goal, budget $\leq$ token cap, categories $\subseteq$ token's | "ACTIVE" mission card, Detail shows full remaining budget                                                    | Mission creation path                          |
| M2 | Category wider than token → rejected             | Mission category not present in the agent's own token categories                         | <code>#missionError</code> — creation refused                                                                | <code>validateMissionAgainstToken</code>       |
| M3 | Currency mismatch → rejected                     | Mission currency $\neq$ token currency                                                   | Creation refused                                                                                             | Currency exact-match rule                      |
| M4 | Budget lower than token cap — allowed and normal | Mission budget e.g. 800 against a token cap of 2000                                      | Created successfully — this is expected, not an error (cumulative vs. per-transaction, different dimensions) | Correct non-conflation of budget types         |
| M5 | Expiry later than token's → rejected             | Mission expiry after the token's own expiry                                              | Creation refused                                                                                             | Expiry bound check                             |
| M6 | Mission-agent binding required                   | Attempt to submit without selecting an agent                                             | Form validation prevents submission                                                                          | Referential integrity                          |
| M7 | Mission selection / detail                       | Click a mission card                                                                     | Mission Detail shows goal, live remaining budget, boundary, expiry, transaction history                      | Read-path correctness                          |
| M8 | Cancellation                                     | <code>POST /missions/:id/cancel</code> (API-level; not a dashboard button)               | Status transitions; a subsequent transaction against it is denied by the mission gate's status check         | <code>checkMissionGate</code> 's status branch |

### 15.3 — Transaction test matrix

| CASE                            | INPUT                                                                               | EXPECTED VERDICT                                      | PIPELINE STOP                                            | LEDGER BEHAVIOR                                                                                        |
|---------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| VALID                           | Within budget, matching category/counterparty/rail                                  | <b>ALLOW</b>                                          | None — full pipeline runs                                | policy_verdict, risk_verdict, decision, execution_result all appended                                  |
| OVER BUDGET                     | Amount > mission's remaining budget                                                 | <b>DENY</b>                                           | Mission gate                                             | mission_policy_verdict + decision appended; no risk/execution entries                                  |
| WRONG CATEGORY                  | Category outside the mission's (or token's) allowlist                               | <b>DENY</b>                                           | Mission gate (if mission attached) else capability layer | Same shape as above, or policy_verdict if capability-layer                                             |
| WRONG COUNTERPARTY              | Counterparty outside the mission's approved list                                    | <b>DENY</b>                                           | Mission gate                                             | mission_policy_verdict + decision appended                                                             |
| WRONG RAIL                      | Rail not in the token's allowed set                                                 | <b>DENY</b>                                           | Capability layer                                         | policy_verdict (allowed:false) + decision appended                                                     |
| EXPIRED / INACTIVE MISSION      | Mission status ≠ "active"                                                           | <b>DENY</b>                                           | Mission gate (status check, first)                       | mission_policy_verdict + decision appended                                                             |
| REVOKED AUTHORITY               | Token or ancestor revoked                                                           | <b>DENY</b>                                           | Capability layer                                         | policy_verdict (allowed:false) + decision appended                                                     |
| SUSPICIOUS AMOUNT               | >3× this agent's own historical mean, 3+ prior transactions                         | <b>ESCALATE</b>                                       | Risk stage (after policy passes)                         | policy_verdict (allowed:true), risk_verdict, decision — no execution_result                            |
| NO MISSION / standing authority | Mission field left as "None"                                                        | Whatever the token alone determines                   | Capability layer directly (no mission gate at all)       | No mission_policy_verdict entry — confirms the mission gate is genuinely skipped, not silently applied |
| EXECUTION FAILURE               | Amount doesn't match mock_x402's fixed catalog price for that counterparty/category | Decision was <b>allow</b> , execution reports failure | After decision, at execution                             | execution_result appended with success:false                                                           |

### 15.4 — Security test cases

| ID | TEST                                       | STEPS                                                              | EXPECTED RESULT                                                                                     | INSPECT                                              |
|----|--------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------|
| S1 | Concurrency attack                         | New attack mission (\$2,000) → Launch attack                       | allowed × 380 ≤ 2000, allowed + blocked = 20                                                        | The four stat numbers, the green banner's spend line |
| S2 | Zero overspend confirmation                | Same run                                                           | "ZERO OVERSPEND — BUDGET HELD" banner, server-confirmed spend = displayed spend                     | Compare the banner's number against the stat row     |
| S3 | Revocation before/after                    | Run scenario                                                       | Before: <b>ALLOW</b> . After: <b>DENY</b> , "revoked" reason                                        | Both verdict lines, rendered at large scale          |
| S4 | Repeated attacks (idempotency under retry) | Launch the attack scenario twice in a row on the same mission      | Second run's budget starts from the first run's already-reduced remaining balance — no double-count | Remaining budget figure between runs                 |
| S5 | Insufficient budget for even one attempt   | Create an attack mission, spend it to near-zero first, then attack | Most/all 20 attempts blocked; still zero overspend                                                  | Allowed count near 0                                 |

### 15.5 — Evidence test cases

| ID | TEST                                    | STEPS                                                    | EXPECTED RESULT                                                                                           |
|----|-----------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| E1 | Clean ledger verification               | Refresh → Verify chain                                   | "✓ HASH CHAIN VERIFIED"                                                                                   |
| E2 | New event appears                       | Submit any transaction, then Refresh                     | New entries at the top with a higher <b>seq</b> , correct <b>prev_hash</b> linking to the prior top entry |
| E3 | Tamper latest entry                     | Tamper latest entry button                               | Header badge → "Ledger tampered"                                                                          |
| E4 | Verification failure after tamper       | Verify chain again                                       | "X INTEGRITY VIOLATION DETECTED", exact entry named                                                       |
| E5 | Hash/signature fields display correctly | Inspect any ledger entry row                             | <b>hash</b> , <b>prev</b> , <b>sig</b> all shown, truncated for readability                               |
| E6 | Verifier pass                           | Export a clean ledger → <b>node verifier/dist/cli.js</b> | Exit code 0, all proofs pass                                                                              |
| E7 | Verifier fail                           | Export a tampered ledger → same command                  | Exit code 1, failure named                                                                                |

## 15.6 — UX / accessibility testing (lighter pass, still required)

| CHECK                                    | METHOD                                                              | PASS CRITERIA                                                                                           |
|------------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| All six nav tabs                         | Click each in sequence                                              | Each renders its own workspace and environment image, no reload                                         |
| Info popovers                            | Hover, then Tab + Enter on each ⓘ                                   | Opens on hover and on keyboard activation; closes on outside click or Escape                            |
| Inputs<br>(text/number/select/textarea)  | Click into each field type across Authority, Missions, Transactions | Focus visible, caret behaves normally, typing registers immediately                                     |
| Keyboard navigation                      | Tab / Shift+Tab through a workspace                                 | Focus order is logical, always visible, never trapped                                                   |
| Buttons                                  | Click every primary action button once                              | All clickable, no dead controls                                                                         |
| Viewport 1920×1080 / 1400×900 / 900×700  | Resize / DevTools device toolbar                                    | No horizontal overflow at any of the three                                                              |
| Backgrounds remain visible               | Visual check at each viewport                                       | Environment photograph clearly visible behind every panel, no full-bleed dark overlay                   |
| Reduced motion                           | Emulate <code>prefers-reduced-motion: reduce</code>                 | App boots and functions with zero console errors; parallax/idle drift stop                              |
| Environment doesn't block pointer events | Click a button directly over a bright part of the background image  | Click registers on the real control underneath — environment layer is <code>pointer-events: none</code> |

PART 16

## 16. Failure / Troubleshooting Guide

| SYMPTOM                                                    | EXPLANATION & FIX                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>localhost</code> refused to connect                  | The server isn't running. Start it: <code>AEGIS_DEMO_MODE=true npm start</code> from the repo root. Wait for "Aegis listening on http://localhost:8787" in the terminal before reloading the browser.                                                                                                                                                     |
| "Missing <code>ANTHROPIC_API_KEY</code> " error on startup | You're <i>not</i> in demo mode. Either set <code>AEGIS_DEMO_MODE=true</code> (recommended for the demo — no key needed) or provide a real key. Never needed for the standard presentation path.                                                                                                                                                           |
| Demo server startup command                                | <code>AEGIS_DEMO_MODE=true npm start</code> — this runs <code>npm run build</code> (compiles TypeScript) then starts the compiled server. First run takes a few seconds longer for the build.                                                                                                                                                             |
| <code>ENOTFOUND</code> / DNS error in a terminal           | This is network/provider connectivity (e.g. a stray external API call, or an unrelated tool trying to reach the internet) — <b>not</b> AEGIS's own logic. AEGIS's demo-mode path makes zero external network calls by design (Part 9). If this appears during a normal demo boot, check your network connection, not the application code.                |
| Invalid API key vs. DNS failure                            | An invalid-key error is an authenticated response from the provider (a 401/403-shaped error, arrives fast). A DNS/ <code>ENOTFOUND</code> failure never reaches the provider at all — it fails before any request goes out, usually instantly, with no HTTP status at all. If in doubt: irrelevant to the demo path, since demo mode makes no such calls. |
| Database stale or already tampered                         | Stop the server, delete the SQLite file at your configured <code>AEGIS_DB_PATH</code> (default <code>./aegis.db</code> , plus any <code>-journal</code> / <code>-wal</code> / <code>-shm</code> siblings), restart. A fresh file is created automatically on next boot.                                                                                   |
| Ledger already shows "tampered" from a rehearsal           | Same fix as above — the tamper demo corrupts the on-disk database directly, so the only clean reset is a fresh database file. Do this as your last rehearsal step before the actual presentation, not your first.                                                                                                                                         |
| Wrong starting principal/agent/mission state               | Sign in with a brand-new principal ID at the sign-in screen — every principal's agents/missions are isolated, so this alone gets you a clean working set without touching the database.                                                                                                                                                                   |
| Port 8787 already in use                                   | Another process (likely a previous, forgotten server run) is bound to it. Find and stop it (Windows: <code>netstat -ano   findstr :8787</code> then stop that PID; macOS/Linux: <code>lsof -i :8787</code> ), or start with a different <code>PORT=...</code> and use that URL instead.                                                                   |
| A workspace's background image is missing/broken           | Confirm <code>public/assets/{overview,authority,missions,transactions,security,evidence}.png</code> all exist and weren't accidentally moved or renamed — the six-workspace crossfade depends on those exact filenames.                                                                                                                                   |
| An unexpected transaction result                           | Open the Decision Inspector's full pipeline trace — it names the exact stage that produced the verdict and the exact reason text. Cross-check against Part 7's scenario table for what that stage's reason string means.                                                                                                                                  |

PART 17

## 17. Pre-Demo Reset Checklist

Run this 10–15 minutes before presenting.

- ☐ Laptop charging / plugged in
- ☐ Browser zoom at 100%
- ☐ Fresh database — delete any existing `aegis.db` (and `journal/wal/shm` siblings) before this final start

- ☐ Server started fresh: `AEGIS_DEMO_MODE=true npm start`, confirm "Aegis listening..." in the terminal
- ☐ Browser open at `http://localhost:8787`, signed in with a clean, memorable principal ID
- ☐ All six workspace tabs clicked once — confirm each background image loads (no broken-image icon)
- ☐ No yellow banner or any leftover dev-mode visual clutter at the top of the page
- ☐ One rehearsal ALLOW transaction fired and confirmed green
- ☐ Database reset again if the rehearsal left state you don't want visible at the real start
- ☐ Security workspace's attack button tested once (confirms the mock merchant/rail is responsive)
- ☐ Evidence tab confirmed clean — "✓ HASH CHAIN VERIFIED," not tampered from a rehearsal
- ☐ Irrelevant windows closed (VS Code, other browser tabs, chat apps)
- ☐ System notifications turned off / Do Not Disturb enabled
- ☐ Backup screenshots or a short screen-recording ready on the laptop, offline, in case live networking fails
- ☐ GitHub repository open in a tab and confirmed it loads (README renders, images display)
- ☐ README's embedded screenshots confirmed loading correctly on GitHub's actual rendering, not just locally
- ☐ No secrets visible anywhere on screen — no terminal scrollbar showing a real key, no `.env` file open
- ☐ This PDF handbook open locally (for a co-presenter to reference, or for your own quick lookup)
- ☐ The exact terminal start command copied somewhere fast to paste (README's Quickstart, or this handbook's Part 16)

## PART 18

## 18. What Features Can Make Us Win

Ranked by technical/demo impact, not marketing appeal.

| # | FEATURE                                       | WHY UNUSUAL / WHY IT MATTERS                                                                                                                                                                                                         | SAY THIS                                                                                                                                               | POINT AT                                                                                     |
|---|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| 1 | Capability attenuation / bounded delegation   | Most systems delegate by copying or widening scope ("give the sub-process the same key"). Here, narrowing is enforced by the token's own cryptographic structure — a wider child cannot be minted, not merely discouraged by policy. | "Authority can only ever narrow going down the chain — that's not a rule we follow, it's a rule the token's own math enforces."                        | The Authority Flow diagram's narrowing rings, struck-through categories on a child agent     |
| 2 | Atomic budget enforcement under concurrency   | Most demo-stage systems have never actually been fired at concurrently — this one has, and holds exactly at the boundary.                                                                                                            | "Twenty requests just competed for the same budget. Zero overspend — not approximately zero, exactly zero."                                            | The per-attempt grid and the green "ZERO OVERSPEND" banner                                   |
| 3 | Immediate ancestor revocation                 | No per-descendant cleanup step exists or is needed — the ancestry check itself is the propagation mechanism.                                                                                                                         | "The transaction didn't change. The authority did."                                                                                                    | The before/after ALLOW—DENY pair, same input                                                 |
| 4 | ALLOW / DENY / ESCALATE separation            | A binary allow/deny model has no place for "policy is fine, but this looks unusual" — escalate is a genuinely distinct, deterministic outcome here, not a UI label on top of deny.                                                   | "Escalate isn't a softer deny. It's what happens when every hard boundary is satisfied and a completely separate signal still says: look at this one." | The Decision Inspector's gold badge with the Intent/Behavioral lines shown separately        |
| 5 | Tamper-detectable evidence                    | Corrupting the database directly still gets caught, immediately, with the exact entry named — most application audit logs offer no such guarantee.                                                                                   | "A log tells you what the system says happened. This lets you check whether that history still agrees with itself."                                    | The status flip from green to red on Evidence                                                |
| 6 | Independent verifier                          | A second, separately-authored tool that shares zero code with AEGIS reaches the same conclusion — this is a materially stronger trust claim than "AEGIS checked itself."                                                             | "This isn't AEGIS grading its own homework."                                                                                                           | The <code>verifier/</code> CLI run in a terminal, if time allows                             |
| 7 | Deterministic demo / production AI separation | The system is architected so a probabilistic model can only ever make an outcome stricter, never grant authority outside the token's own bounds — a real safety property, not a caveat.                                              | "Judgment may assist risk evaluation. It never gets to decide the budget."                                                                             | The Decision Inspector's <code>Intent:</code> line naming itself as a deterministic stand-in |

### THE TOP THREE TO EXPLAIN EXTREMELY WELL

**1. Attenuation** (it's the conceptual foundation everything else depends on) · **2. Atomic budget enforcement under concurrency** (it's the most viscerally convincing live demo moment) · **3. Tamper detection** (it's the emotional and technical climax — a status flipping red on real, unstaged data is hard to argue with). If time runs short anywhere in the script, protect these three before anything else.

## PART 19

## 19. Questions Judges May Ask

### "What prevents the agent from just bypassing AEGIS?"

In this architecture, nothing prevents a rail credential from being called directly by something that isn't AEGIS — AEGIS is the decision layer, not a network firewall. What it does guarantee is that *if* a rail adapter only ever receives credentials scoped to it and is only ever invoked through AEGIS's own execution path, no valid capability token exists that could authorize the bypassed path either.

Precisely: AEGIS never holds the rail's own settlement credentials for the agent's use directly — the agent has no rail credential to use standalone. Full production-grade isolation of "the agent literally cannot reach the rail except through AEGIS" is an infrastructure/network-boundary concern, named in `docs/SYSTEM_ARCHITECTURE.md` §7 as separation of decision and custody, not fully solved at this MVP stage.

Don't say: "It's impossible." Say: "the agent is never handed a credential that lets it bypass this."

### "Why not just use API keys?"

An API key answers "who is this," not "what, exactly, up to how much, is this specific request allowed to do." A capability token encodes the boundary itself, cryptographically — the check doesn't depend on trusting an application's own bookkeeping of what that key is "supposed to" be limited to.

A leaked or over-trusted API key can do anything the backend permits for that key's role. A leaked capability token can do exactly what its own caveats say, and revoking it takes effect immediately on the next verification.

### "How is this different from RBAC?"

RBAC assigns a role, and the role's permissions live in a central table someone can edit. Here, the permission set travels *inside* the credential itself, is mathematically bounded by its parent at mint time, and needs no central lookup to verify.

RBAC roles are typically coarse and shared across many principals; attenuation produces a unique, narrower-every-time token per delegation, verifiable offline from the token alone (plus revocation state).

### "Why use capability tokens specifically?"

Because the property we need — "this credential can only ever narrow as it's delegated further" — is exactly what capability-based security with attenuation is designed to guarantee, and Biscuit gives us that with real cryptographic enforcement rather than a convention.

### "What happens if the agent is compromised?"

Its blast radius is exactly its token's own bounds — the max amount, categories, and rails it was actually delegated, nothing more — and it can be revoked instantly, cutting off it and everything it delegated further.

Don't say: "nothing bad can happen." Say: "the damage is capped by design, and stoppable instantly."

### "What prevents budget race conditions?"

One atomic SQL statement that checks and reserves budget in the same write — see Part 10. Demonstrated live under 20 genuinely concurrent requests.

### "What does ESCALATE actually do?"

Stops execution, exactly like deny, but for a different reason: every hard policy boundary passed, and a separate risk signal (behavioral anomaly, or an inconsistent/ambiguous intent judgment) flagged it anyway. See Part 8.

Don't imply a human-approval workflow exists to click "approve" on an escalated transaction — that's not built; escalate currently means "returned, not executed."

### "Where is AI used?"

In the intent-consistency judge only — comparing an agent's delegated goal against a transaction's stated purpose. In this demo it's a disclosed deterministic stand-in; the real Anthropic-backed version exists in code and requires a credential we're not using for this presentation. See Part 9.

### "Why Anthropic?"

It's the model provider the risk-judge interface is built against in this codebase; the interface itself is pluggable, so swapping providers is an adapter, not an architecture change.

### "Why not use the LLM for all policy decisions?"

Because a model's judgment is probabilistic and can be wrong in either direction. Budget, category, and revocation enforcement are deterministic code specifically so a bad or manipulated model output can, at worst, cause an unnecessary escalation — never an unauthorized spend.

### "What happens if Anthropic is down?"

`safeJudge()` treats a judge failure/timeout as `"unavailable"`, which `combineRiskSignals()` maps straight to `escalate` — fail-safe, not fail-open. Demo mode is unaffected either way, since it never calls a real model.

### "Is real money moving?"

No. This demonstration runs in `AEGIS_DEMO_MODE=true`, which registers only a self-built mock payment rail — no real rail is reachable in this mode regardless of configuration.

### "Where is Razorpay in this?"

Razorpay is the buildathon this was built for — there is no Razorpay API integration in the current codebase. The rail layer is adapter-based by design (a Stripe test-mode adapter and a self-built mock adapter both exist today); adding a Razorpay adapter is architecturally the same shape of work as either of those, not a redesign.

Never say "we integrate with Razorpay" — we don't, and shouldn't imply it.

### "Is this production-ready?"

No, and we say so directly — this is a from-scratch technical demonstration. It hasn't been load-tested at real scale, integrated with a real agent framework, or exposed to real users or real money. See `docs/SECURITY_MODEL.md`'s limitations section for the full, specific list.

### "Can authority be widened?"

Not through attenuation — structurally impossible, enforced before a wider child token is ever minted. The only way to grant more authority is to mint an entirely new, independent root token.

### "What happens if a parent is revoked?"

Every descendant fails its next verification — see Part 11.

### "Can the database be tampered with?"

Yes, directly, at the storage layer — we don't claim otherwise. What we claim, and demonstrate live, is that such tampering becomes detectable, both by AEGIS's own check and by a completely independent verifier.

### "Is the ledger a blockchain?"

No. It's a hash-chained, Ed25519-signed append log with one trusted signer (the running process's own key) — tamper-evident, not distributed consensus. We're explicit about this distinction in our own docs.

### "Who verifies the verifier?"

Its own test suite — 58 tests, run independently of the main suite — including adversarial cases where it must correctly *refuse* to certify a tampered or incomplete export rather than guessing. Anyone can also read its ~150 lines of core logic themselves; it deliberately imports nothing from AEGIS to keep that read small.

**"Can someone steal a capability token?"**

A stolen token can be used up to exactly its own bounds until revoked — same category of risk as a stolen scoped credential in any capability system. Revocation is instant once detected. Full token-storage hardening (e.g. how a real agent framework would store these client-side) is outside this MVP's scope.

**"How would you deploy this in production?"**

The application itself needs no architecture change — it's a normal long-running Node/Express process. `render.yaml` in this repo is a verified-locally Render Blueprint for exactly that; it has not been deployed to a public URL. Production hardening beyond deployment — HSM-backed signing keys, real credentialed rails, load testing — is explicitly named as future work, not claimed as done.

**"What's the biggest limitation?"**

Be honest, pick one that's actually true and interesting: tail truncation of the ledger (deleting the most *recent* entries) is not detectable from a single snapshot — there's nothing after the cut to contradict. Or: no real AI risk evaluation or real payment has been exercised in this environment, by design, for this demo.

**"What's actually novel here?"**

Not any single primitive alone (capability tokens, hash chains, and atomic reservations each exist elsewhere) — the composition: authority that provably narrows through delegation, bound again to a mission, checked through a pipeline that keeps deterministic boundaries out of AI's hands, revocable instantly, and independently verifiable afterward, all in one coherent, working system.

PART 20

## 20. What NOT to Say During the Demo

**DO NOT SAY** — "We built an unhackable payment system."

Say instead: "We built a system where the budget and authority boundaries are enforced deterministically and provably, under real concurrent load."

**DO NOT SAY** — "This is blockchain."

Say instead: "This is a hash-chained, signed ledger — tamper-evident, with one trusted signer. We're precise about that distinction on purpose."

**DO NOT SAY** — "The AI guarantees safety."

Say instead: "AI may assist risk judgment. The budget and authority boundaries are deterministic code, not a model's opinion."

**DO NOT SAY** — "We have Razorpay integration."

Say instead: "The rail layer is adapter-based — we have a Stripe test-mode adapter and a self-built mock adapter today; a Razorpay adapter is the same shape of work."

**DO NOT SAY** — "This is production-ready."

Say instead: "This is a from-scratch technical demonstration of the core trust architecture — here's specifically what production hardening would still require."

**DO NOT SAY** — "Nothing can go wrong / it's impossible to bypass."

Say instead: name the actual guarantee precisely, and name the actual limitation next to it — see Part 19's answers for the pattern.

PART 21

## 21. Presentation Style & Opening

Do not open with "Good morning judges, my name is..." — that's a formality, not a hook. Open with the actual tension the whole product resolves, so the room looks up in the first sentence instead of the fifth.

**THE OPENING — MEMORIZE THIS, DON'T READ IT**

*"What happens when an AI agent doesn't just recommend a purchase — but actually has the authority to make it? Authentication tells you who the agent is. It doesn't tell you whether this specific \$380 charge, right now, is something it was actually allowed to do. That gap — between being authenticated and being authorized, precisely, for one action — is where AEGIS lives. It exists between an agent's intent and its execution."*

*Three words: **bounded, revocable, verifiable**. That's the whole system. I'll show you all three, live, against real code, in the next seven minutes."*

Not theatrical, not cringe — just precise tension stated plainly, then resolved by what you're about to show, not by adjectives.

## PART 22 — THE MOST IMPORTANT SECTION

## 22. Final 7–8 Minute Master Demo Script

Target total: 7:00–8:00. Rehearsed and timed against the real application — see Part 25 for verification notes. Pre-stage a fresh database and a signed-in principal before the room fills (Part 17); the clock below starts once you're already looking at the Overview screen.

### 00:00 – 00:40 — Opening (no click)

| SAY                                                | DO                                  | POINT AT                            | JUDGE SHOULD UNDERSTAND                                                   |
|----------------------------------------------------|-------------------------------------|-------------------------------------|---------------------------------------------------------------------------|
| The opening line, Part 21, verbatim or close to it | Stand, don't touch the keyboard yet | Nothing — eye contact with the room | This is a precise technical problem, not a generic "AI is powerful" pitch |

### 00:40 – 01:20 — Act 1: Overview

| SAY                                                                                                       | DO                        | POINT AT                          | JUDGE SHOULD UNDERSTAND                                     |
|-----------------------------------------------------------------------------------------------------------|---------------------------|-----------------------------------|-------------------------------------------------------------|
| "AEGIS is a trust-control layer for autonomous agents that spend money — bounded, revocable, verifiable." | Overview tab already open | The hero statement, the fact grid | The whole thesis is visible before a single action is taken |

### 01:20 – 02:20 — Act 2: Authority

| SAY                                                                                                  | DO                                                   | POINT AT                                                                   | JUDGE SHOULD UNDERSTAND                              |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------|
| "Authority isn't binary — it flows through a chain, and it can only narrow going down, never widen." | Click Authority                                      | The <code>Root → Delegated → Attenuated → Mission bound</code> chain label | The mental model, before any data exists             |
| "Here's a root agent with a \$2,000 cap across flights, hotels, software."                           | Create root agent (pre-typed values ready to submit) | The new card in the tree                                                   | Authority starts as a real, bounded credential       |
| "Now I delegate a sub-agent — and it can only get narrower."                                         | Select → Attenuate → Create (\$800, flights only)    | "Delegated (attenuated)," the struck-through categories                    | Narrowing is enforced by the token, not a convention |

### 02:20 – 03:00 — Act 3: Mission

| SAY                                                                                                                                               | DO                                                                                                        | POINT AT                                           | JUDGE SHOULD UNDERSTAND                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------------|
| "Even valid authority isn't enough — the agent also needs a bounded mission. It doesn't get money, it gets permission to complete one objective." | Click Missions → create (agent = the sub-agent, budget 800, category flights, counterparty acme-airlines) | "Budget — the boundary this mission cannot exceed" | A second, independent, narrower boundary layered on top |

### 03:00 – 04:30 — Act 4: Transactions — ALLOW / DENY / ESCALATE

| SAY                                                                                                                                                                                        | DO                                                                                                                            | POINT AT                                                                                                                             | JUDGE SHOULD UNDERSTAND                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| "Every execution goes through a real decision pipeline — the agent never directly converts intent into money moving."                                                                      | Click Transactions                                                                                                            | The <code>Mission → Capability → Policy → Risk → Decision → Execution</code> chain                                                   | Multiple independent checks, not one if-statement                                         |
| "\$380, a real flight purchase, within every boundary."                                                                                                                                    | Fill form, Execute                                                                                                            | Green <code>ALLOW</code> , all-green pipeline, the settlement reference                                                              | A genuinely valid case succeeds end-to-end                                                |
| "Now the same agent tries to massively exceed its mission's budget."                                                                                                                       | Amount 5000, Execute                                                                                                          | Red <code>DENY</code> , the mission-gate stage highlighted                                                                           | A denial stops exactly where policy actually stopped it                                   |
| "And here's the one most people don't expect — an escalation. This agent has a small, consistent spending pattern; I'm about to submit something wildly out of line with its own history." | Switch to the pre-warmed escalate-agent (3 × \$50 already submitted before the room filled — see the tip below); submit \$600 | Gold <code>ESCALATE</code> , the "12.0x this agent's historical average" reason, the <code>Intent: consistent</code> line next to it | A deterministic, disclosed behavioral signal — not AI, and stated as such right on screen |

**Rehearsal tip, honestly disclosed as prep, not live fabrication:** submitting the three \$50 warm-up transactions for the escalate-agent *before* the room fills (during Part 17's checklist) saves ~30 real seconds of live clicking without changing what the room sees — the \$600 transaction and its verdict are still fired live, on real data, in front of them.



**04:30 – 06:00 — Act 5: Security — concurrent attack + revocation**

| SAY                                                                                                                        | DO                                                                              | POINT AT                                                     | JUDGE SHOULD UNDERSTAND                                         |
|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------|
| "Limits are easy to enforce sequentially. The real question is whether they survive contention."                           | Click Security, select an agent with headroom, New attack mission (\$2,000)     | The panel heading                                            | Setting up a genuine adversarial condition, not a staged number |
| (pause, click, then:) "Twenty requests just competed for the same budget. Five allowed, fifteen blocked. Overspend: zero." | Launch attack (20 × \$380), wait ~5s for resolution                             | The per-attempt grid, then the green "ZERO OVERSPEND" banner | An atomic database write, not client-side pacing, decided this  |
| "Now watch trust get revoked, live. This transaction works right now —"                                                    | Run the revocation scenario's first half (or a fresh Execute on the same agent) | The "before" green verdict                                   | Authority is currently valid                                    |
| "— and after I revoke it, the exact same request —"                                                                        | Revoke → repeat the transaction                                                 | The "after" red verdict, "revoked" reason                    | "The transaction didn't change. The authority did."             |

**06:00 – 07:20 — Act 6: Evidence — verify, tamper, verify**

| SAY                                                                                                                                                                                 | DO                                    | POINT AT                                              | JUDGE SHOULD UNDERSTAND                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|-------------------------------------------------------|---------------------------------------------------------------|
| "AEGIS doesn't just control what an agent can do — it preserves evidence of what happened. Let's check it."                                                                         | Click Evidence, Refresh, Verify chain | Green "✓ HASH CHAIN VERIFIED"                         | A real, checkable integrity state — not a static label        |
| "Now I'm going to tamper with the database directly — bypassing AEGIS entirely, the way an attacker with raw storage access would."                                                 | Click Tamper latest entry             | (pause a beat)                                        | This is a genuine attack simulation, not a scripted animation |
| "Verify again —" (let the red status land, then, deliberately:) "A log tells you what the system says happened. This lets you check whether that history still agrees with itself." | Click Verify chain                    | Red "X INTEGRITY VIOLATION DETECTED," the named entry | Tampering is detected, not merely logged as suspicious        |

**07:20 – 07:50 — Closing**

| SAY                                                                                                                                                                                                                                                                                                                           | DO                          | POINT AT                                                                  | JUDGE SHOULD UNDERSTAND                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------|
| "Autonomous agents don't need unlimited trust to be useful. They need exactly enough authority to complete a mission, the ability to lose that authority instantly, and evidence we can verify afterward. That's AEGIS. Bounded. Revocable. Verifiable. We don't ask the agent to behave — we make the boundary enforceable." | Step back from the keyboard | The Overview screen's hero statement, if you have time to flip back to it | The thesis, restated, now backed by everything just shown |

PART 23

**23. 2-Minute Emergency Version**

Use when judges cut the slot short. Six moves, no filler.

- Problem (15s):** "Authentication tells you who an agent is, not what it's allowed to do, right now. AEGIS answers that second question — bounded, revocable, verifiable."
- Authority narrowing (25s):** Authority tab, attenuate one sub-agent live, point at the struck-through categories. "Authority can only narrow going down — enforced by the token, not a convention."
- One transaction (20s):** Transactions tab, one ALLOW. "Every execution goes through a real pipeline before anything moves."
- Concurrent attack OR revocation — pick whichever is faster to reach from current state (35s):** Attack: "Twenty concurrent requests, zero overspend." Revocation: "The transaction didn't change. The authority did."
- Tamper detection (20s):** Evidence tab, Verify → Tamper → Verify. "A log tells you what happened. This lets you check whether that history still agrees with itself."
- Closing (10s):** "Bounded. Revocable. Verifiable. That's AEGIS."

PART 24

## 24. 30-Second Elevator Version

*"AI agents are starting to spend real money. AEGIS gives them a cryptographic permission slip instead of a password — one with a spending cap and an expiry baked in, that can only get narrower as it's delegated further, revoked instantly with the revocation actually sticking, and every decision recorded in a way a completely separate tool can verify without trusting AEGIS at all. Bounded. Revocable. Verifiable."*

PART 25 — PRESENTER CHEAT SHEET

## 25. Final One-Page Cheat Sheet

### OPENING LINE

"What happens when an AI agent doesn't just recommend a purchase — but actually has the authority to make it? Authentication tells you who the agent is. It doesn't tell you whether this specific charge, right now, is allowed."

### 3 CORE IDEAS

1. Authority narrows, never widens, as it's delegated. 2. A boundary only matters if it survives concurrency and revokes instantly. 3. Evidence must be independently checkable, not just logged.

### TOP 3 WINNING FEATURES (DEFEND THESE HARDEST)

1. Attenuation (the foundation) · 2. Atomic budget enforcement under concurrency (the viscerally convincing moment) · 3. Tamper detection (the climax)

### DEMO ORDER

Overview → Authority (create + attenuate) → Missions (create) → Transactions (ALLOW → DENY → ESCALATE) → Security (attack → revoke) → Evidence (verify → tamper → verify) → close

### ALLOW INPUT

380 · flights · mock\_x402 · acme-airlines · mission attached, within budget

### DENY INPUT

5000 against an \$800 mission — or the same transaction after revoking its agent

### ESCALATE INPUT

3 prior transactions at \$50 (pre-warm before the room fills), then one at \$600 (12× the mean, over the 3× threshold), no mission, category software

### ATTACK EXPECTATION

$20 \times \$380$  vs. \$2,000 budget → allowed  $\times 380 \leq 2000$ , allowed + blocked = 20, "ZERO OVERSPEND — BUDGET HELD"

### REVOCATION STEPS

Execute (ALLOW) → Revoke agent → Execute again (DENY, "revoked" reason)

### TAMPER STEPS

Verify chain (green) → Tamper latest entry → Verify chain (red, entry named)

### FINAL LINE

"Bounded. Revocable. Verifiable. We don't ask the agent to behave — we make the boundary enforceable."

### 5 MOST LIKELY JUDGE QUESTIONS

1. "Where is AI actually used?" 2. "Is real money moving?" 3. "Is the ledger a blockchain?" 4. "What prevents budget race conditions?" 5. "Is this production-ready?" — full answers in Part 19.